

Preparacion Examen I

TEMAS BLOQUE A

Registro y Archivos Binarios

Secuencia de pasos a seguir

- Seleccionador de archivo a guardar
 - Apertura de entrada de datos
 - Escritura
 - Registro en archivo
-
- Para trabajar con archivos en c++, en el caso de registrar datos, se hace uso de la libreria fstream, la cual nos da las funciones de

```
std::ofstream out(file name, type open) //funcion para la entrada de datos
```

```
//Caso de aplicacion
```

```
std::ofstream out("registros.dat", std::ios::binary | std::ios::app);
```

- Para el registro de datos en un archivo binario, existen dos formas: **una forma directa, y la forma de la serializacion mediante un buffer**. Ambas formas serializan datos, pero tienen propositos y tecnicas distintas.
 - **Serialización:** es el proceso de convertir el estado de un objeto o estructura de datos a un formato lineal de bytes para ser almacenados.

Forma directa, uso de un struct

Propósito

- Esta forma es ideal cuando **se trabaja con archivos de tamaño FIJO**

Tecnica

- Consiste en crear un struct, el cual contenga como atributos la información a almacenar.
- En estos casos, para manejar lineas de texto, como string, se crean arreglos de chars con tamaño fijo. Significando que conocemos el tamaño del texto de antemano
- Podemos sufrir de padding en caso que quede espacio sin usar.
 - ¿Cómo tratar esto?
 - Crear los datos en la misma funcion en caso que no los tenga

```
//Creacion de struct
struct registroVuelo{
    int numVuelo;
    int cantPasajeros;
    char codigoAvion[5];
};

void registrarVuelo(std::string serieVuelos, registroVuelo& registro){

    std::ofstream out(serieVuelos, std::ios::binary | std::ios::app);

    //verificacion de archivo
    if(!out.is_open()){
        std::cerr << "No se pudo abrir archivo";
        return;
        //return -1 en caso de ser int
    }
```

```

    /*
        En caso de querer escribir esto en la misma func
ion
        registro.numVuelo= 16;
        registro.cantPasajeros=600;
        registro.codigoAvion = "XCK68";
    */

    out.write(reinterpret_cast<char *>(&registro), sizeof
f(registro));
    out.close();
}

```

- Observaciones:
 - En caso de querer escribir al final del archivo, se usa la bandera `std::ios::app`....esta garantiza esto
 - Por defecto, el ofstream crea el archivo indicado en caso que no exista.
 -

Registro de archivos con tamaños variables, uso de un buffer

Proposito

- Esta técnica es util cuando ya nuestros archivos no tienen un tamaño fijo, si no que tienen elementos con tamaños variables, como por ejemplo, el `std::string`

Tecnica

- Se utiliza una estructura `std::vector` de `chacrs` para colocar todos los datos a manejar

- El buffer maneja un puntero interno, el cual nos ayuda a manejar los saltos en memoria
 - Parecido a como se manejaba el puntero en java
- Para llevar control del tamaño de los datos de tamaño variados ingresado, antes de colocarlos en el buffer, se coloca su tamaño
 - Así podemos conocer su extensión y luego mover el puntero interno del buffer de una manera más adecuada
- Al final, el contenido de este buffer es escrito directamente al archivo

```
//=====CLASE BUFFER.hpp=====

#pragma once //esto indica que solamente se instancia un
a vez. Si se vuelve a llamar, no se hace otra instancia
#include <vector>
#include <string>

class Buffer{
    //constructor
    Buffer(size_t size) : buff(size), p(buff.data()) {}
private:
    std::vector<char> buff;
    char *p;

public:
    //Funciones auxiliares

    char *data(){
        return buff.data();}

    size_t size(){
        return buff.size();}

    //Se hacen las funciones de escritura, tanto de
    datos primitivos como de datos ya mas
```

```

        //complejos, como string, mediante el uso de Tem
        plate Overloading

        //Unificacion de metodos write(int, unsigned, fl
        oat, double) mediante template
        template <typename T>
        void write(T value){
            *reinterpret_cast<T*>(p) = value; //serializ
            acion de valor
            p+=sizeof(T); //movimiento de puntero
        }

        //Unificacion de metodo write(string) para mante
        ner como vector de chars
        template <typename It>
        void write (It beg, It end){
            std::copy(beg, end, p);
            p+= std::distance(beg, end);
        }
    }

```

- En un escenario practico, solamente se crea una instancia del buffer y podemos guardar los datos. Recordando que para aquellos datos de tamaño variable, se coloca primero su tamaño a forma de bandera.

```

//===REGISTRO DE VUELO UTILIZANDO DATOS VARIADOS===
#include <string>
#include "Buffer.hpp" //importamos clase buffer local como
ejemplo
#include <fstream>
#include <iostream>

struct registroVuelo{
    int numVuelo;

```

```

    int cantPasajeros;
    std::string nombreCapitan;
    std::string modeloAvion;
};

void registrarVuelo(std::string nameFile, registroVuelo &r
egistro){
    std::ofstream out(nameFile, std::ios::binary | std::io
s::app);

    if(!out.is_open()){
        std::cerr <<"File not opened";
        return;
    }

    //determinamos el tamaño del registro
    size_t sizeReg = sizeof(int) + sizeof(int)
        + registro.nombreCapitan.length() + registro.model
oAvion.length();

    Buffer buffer(sizeReg);
    buffer.write(registro.numVuelo);
    buffer.write(registro.cantPasajeros);
    buffer.write(registro.nombreCapitan.length());
    buffer.write(registro.nombreCapitan.begin(), registro.
nombreCapitan.end());
    buffer.write(registro.modeloAvion.length());
    buffer.write(registro.modeloAvion.begin(), registro.mo
deloAvion.end());

    out.write(buffer.data(), buffer.size());
    out.close();
}

```

Lectura Secuencial de Registro con Acumulacion

- La lectura de datos de un archivo binario lleva la misma mecanica que el registro de los mismos. Dependiendo del tipo de registro, fijo o variado, existen tecnicas especificas para cumplir con estas funciones

Lectura de archivos de tamaño fijo

- Para este caso, se hace uso del `std::ifstream` en combinacion de un ciclo `while`, para leer los varios bloques de datos que existan en el archivo
 - Esto es capaz debido a que sabemos que cada registro tiene un tamaño fijo

```
//=====LECTURA DE ARCHIVO BINARIO DE TAMANIO FIJO=====
#include <fstream>
#include <iostream>
#include <string>

struct registroVuelo{
    int numVuelo;
    int cantPasajeros;
    char codigoAvion[5];
};

void readRegistro(std::string fileName, registroVuelo &registro){
    //apertura en modo lectura
    std::ifstream in(fileName, std::ios::binary);
    //revision
    if(!in.is_open()){
        std::cerr <<"FILE NOT OPENED";
        return;
    }

    registroVuelo reg ={};

    //Nos movemos bloque por bloque para extraer la in
```

```

fo
    while(in.read(reinterpret_cast<char*>(&reg), sizeof
f(registro))) {

        //Impresion de informacion extraida
        std::cout << "Numero Vuelo: " << reg.numVuelo << "\n";

        std::cout << "Cantidad Pasajeros: " << reg.cantPasajeros << "\n";

        std::cout << "Codigo de Avion: " << std::string (reg.codigoAvion, 5) << "\n";

        std::cout << "-----\n";

    }
    in.close();
}

```

Lectura de archivos con datos de tamaño variable

- Para extraer estos datos, se hace uso de las "banderas" que almacenaban el tamaño de los string.
 - Estos valores nos permitirán luego usarlas para crear un string específico para el almacenaje de la información que vamos a extraer.

```

//=====LECTURA DE ARCHIVO BINARIO EN TAMANIO VARIABLE=====
=====
#include <iostream>
#include <fstream>
#include <string>

struct registroVuelo{
    int numVuelo;
    int cantPasajeros;
    std::string nombreCapitan;
    std::string modeloAvion;
}

```



```

};

void readRegistro(std::string fileName, registroVuelo &registro){
    std::ifstream in(fileName, std::ios::binary);
    if(!in.is_open()){
        std::cerr<< "FILE NOT OPENED";
        return;
    }

    int numVuelo;
    int cantPasajeros;
    int sizeCaptainName;
    int sizePlaneModel;

    in.read(reinterpret_cast<char*>(&numVuelo), sizeof(int));
    in.read(reinterpret_cast<char*>(&cantPasajeros, sizeof(int));

    //Lectura de banderas
    in.read(reinterpret_cast<char*>(&sizeCaptainName), sizeof(int));
    std::string captainName(sizeCaptainName, '\\0');
    in.read(&captainName[0], sizeCaptainName);

    in.read(reinterpret_cast<char*> (&sizePlaneModel), sizeof(int));
    std::string planeModel(sizePlaneModel, '\\0');
    in.read(&planeModel[0], sizePlaneModel);

    std::cout<< "Numero Vuelo: "<<numVuelo <<"\n"
        <<"Cantidad de Pasajeros: "<<cantPasajeros <<"\n"
        <<"Nombre de Capitan de Vuelo: "<<captainName <
        <<"\n"
        <<"Modelo de Avion: "<<planeModel<<"\n"

```

```

        <<"-----";

    in.close();
}

```

Representacion de Enteros con Signo (Complemento a Dos)

Serializacion con Clase Buffer

- Con respecto al manejo de los offsets, es importante no confundirlos con el tamaño que ocupan en memoria los datos.
 - Es decir, para manejar el offset de donde empieza cada dato, simplemente se determina mediante la suma acumulativa de los tamaños que ocupen cada dato.
 - Si tenemos datos de 8 , 50, 4, 8....todo el bloque de dato tiene un tamaño de 70 bytes.
 - El calculo de offset, para saber en donde empieza el siguiente dato, es realizar una suma acumulativa, empezando siempre desde el 0
 - $0+8 = 8$ / $8 + 50 = 58$ / $58 + 4 = 62$ / $62 + 8 = 70$
 - Si se debe que determinar lo que ocupa cada dato, ahi si se analiza con un -1 en el indice
 - Es decir, para saber de donde a donde ocupa todo el bloque de datos, se sabe que este está ocupando espacio desde 0 al 69.
- Manejar una clase buffer cuando dejamos ya asignados espacios designados, como un arreglo de characters, es importante ya que nos permite que no nos ocurra un offset indeseado con el resto de datos, como cuando utilizamos alguna función genérica para escribir texto
- La instrucción en C++ que permite persistir todo el contenido del buffer a la memoria secundaria, en tan solo una operación, se logra mediante

```
out.write(buffer.data(), buffer.size()); //asumiendo que ya hay una clase buffer con estas funciones
out.write(reinterpret_cast<const char*>(buffer.data()), buffer.size());
```

- Una función para la lectura de datos, utilizando las templates se logra mediante:

```
template <typename T>
T Buffer::read(){
    T value = reinterpret_cast<T*>(p);
    p+= sizeof(T);
    return value;
}
```

Endianness y Almacenamiento de Enteros

Endianness en La Lectura de Archivos (Efecto Practico)

Considere que un archivo binario fue escrito en un sistema que utiliza formato **Big-Endian**. En dicho archivo se guardó un registro que contiene un entero de 32 bits (`uint32_t id`) con el valor `42` y un arreglo de caracteres (`char model[10]`).

Posteriormente, este archivo binario es leído directamente desde el disco por un programa ejecutado en una arquitectura **Little-Endian** sin realizar ningún tipo de conversión de bytes.

Preguntas

- Explique qué sucede con el orden de los bytes en memoria al leer el entero `id`.
 - Dado que estamos manejando un formato Big-Endian, el orden de los bytes se escribirán de una forma natural, en un orden de derecha a izquierda, con el byte más significativo en el byte 0.

- A la hora de ser leído por el programa un formato Little-Endian, este interpreta el byte 0 como el byte menos significativo, cambiando el orden hexadecimal por completo.
- Muestre la representación hexadecimal de `42` almacenado en **Big-Endian** (4 bytes).
 - Big Endian → 0x0000002A
 - Little Endian → 0x2A000000
- Determine el valor entero en base 10 que observará la máquina **Little-Endian** al interpretar esos mismos bytes.
 - A la hora de leer estos bytes en un formato Little-Endina, en vez de leer el numero 42, se interpretara como el numero 704, 643, 072
- Explique qué sucede con el contenido del arreglo de caracteres `char model[10]` al ser leído en la máquina **Little-Endian**. ¿Sufre alguna alteración en el orden de sus elementos? Justifique su respuesta.
 - Este no sufre ninguna alteracion. Esto debido a que los formatos Little-Endian o Big-Endian solamente afectan a datos de tipo numerico (int, float, double, short). En un arreglo de caracteres(char[] o uint8_t[]), cada caracter es leído de forma independiente.
- Escriba un fragmento de código en C++ que utilice una `union` para detectar si el sistema donde se ejecuta es **Little-Endian** o **Big-Endian**.
- Su solución debe incluir la definición de la `union` (por ejemplo, combinando un entero de 32 bits y un arreglo de 4 bytes).
- Indique la condición lógica exacta que determina qué tipo de arquitectura es.

```
union endiannessDetector{
    uint32_t i;
    uint8_t b[4];
}

bool isEndian{
    endiannessDetector end;
    end.i=1;
}
```

```

    return (end.b[0]==1);
}

```

- Aquí, la union es una estructura de datos que, así como el struct, puede almacenar varios datos, sin embargo, esta se caracteriza ya que guarda todos los datos bajo una misma dirección en memoria. Todos trabajan en el mismo espacio y van trabajando "uno por encima de otros".
 - Dado que todos los datos trabajan bajo el mismo espacio en memoria, esto permite la reinterpretación de los datos sin necesidad de hacer cambios bruscos
- El uso de datos `uint32_t`, simplemente son datos de un tamaño fijo, para evitar cambios en base al compilador.

Portabilidad de Formatos Binarios (Caso Aplicado)

- Para realizar una conversión de datos almacenados en Little Endian a big endian, se debe realizar ciertos pasos para asegurar una correcta conversión
 - Paso 1 -Desarrollar una estructura que almacene datos de tipo fijo (`uint32_t` y `uint16_t`). Esto ya que evitaremos las variaciones de tamaño que pueden tener los `int`, `double`, `float` en cada procesador

```

//PASO 1 -Crear una estructura almacenadora con datos d
e tamaño fijo
struct Record{
    uint32_t id; //4 bytes exactos
    uint16_t code; //2 bytes exactos
    uint8_t status; //1 byte exacto
};

```

- Paso 2 → Desarrollar funciones para convertir los datos de 4 bytes (32 bits) y de 2 bytes (16 bits).

- Esto se logra mediante el *bit-shifting* logrado por los operadores >>. Asimismo, se rellena los espacios anteriores con ceros mediante 0x000000FF, y similares
- Se manejan funciones distintas para datos de 4 bytes y 2 bytes, esto para evitar desplazamientos de mas.

```
//PASO 2 -Desarrollo de funciones para convertir a big Endian
//tanto para datos de 4 bytes como de 2 bytes
uint32_t toBigEndian32(uint32_t val){
    //Conversion a Big Endian mediante bit-shifting
    return ((val >> 24) & 0x000000FF) |
           ((val >> 8) & 0x0000FF00) |
           ((val << 8) & 0x00FF0000) |
           ((val << 24) & 0xFF000000);
}

uint16_t toBigEndian16(uint16_t val){
    return ((val >> 8) & 0x00FF) |
           ((val << 8) & 0xFF00);
}
```

- PASO 3 → Antes de escribir los datos en un archivo, estos se deben que convertir primero a su formato BIG ENDIAN
- PASO 4 → Se escriben los datos, uno por uno, directamente en el archivo mediante el sizeof(tipo de dato), y no escribiendo toda la estructura de un solo. Esto para evitar el posible padding

```
//Desarrollo de Funcion conversora
void guardarRegistroBE(const std::string &fileName, const Record& reg){
    std::ofstream out(fileName, std::ios::binary);
    if(!out.is_open()) return;

    //PASO 3 -Convertir datos a BIG ENDIAN antes de esc
```

```

ribirlos en archivo
    uint32_t id_BE = toBigEndian32(reg.id);
    uint16_t code_BE = toBigEndian16(reg.code);
    uint8_t status_BE = reg.status; //Los datos de 1 byte no ocupan conversion

    //PASO 4 -Escritura directa en archivo, evitando asi cualquier padding
    out.write(reinterpret_cast<const char*>(&id_BE), sizeof(id_BE));
    out.write(reinterpret_cast<const char*>(&code_BE), sizeof(code_BE));
    out.write(reinterpret_cast<const char*>(&status_BE), sizeof(status_BE));

    out.close();
    std::cout<< "AVISO: CONVERSION A BIG ENDIAN EXITOSA";
}

```

TEMAS BLOQUE B

Archivos de texto

- Aparte de los archivos binarios, tambien se ha trabajado, de forma introductoria, pero tampoco no explotada, el manejo de archivos de texto.
- Así como con los archivos binarios, es importante conocer como hacer las funciones principales: la lectura y la escritura

Escritura en archivos de texto

- Para lograrlo de forma directa y sencilla, se utiliza tambien el std::ofstream de la libreria fstream.

- Para cada linea a escribir, se hace mediante el uso de "<<"

```
//=====ESCRITURA EN ARCHIVO DE TEXTO=====
#include <fstream>
#include <iostream>
#include <string>

int main(){
    std::ofstream agenda("agenda.dat", std::ios::app);

    if(!agenda.is_open()){
        std::cerr << "FILE NOT OPENED";
        return -1;
    }

    agenda << "ESTO ES UN TEXTO EN LINEA 1"<< std::endl;
    agenda << "ESTO ES UN TEXTO EN LINEA 2"<< std::endl;

    agenda.close();
    return 0;
}
```

Lectura en archivos de texto

- Para lograr esto, la tecnica mas adecuada es mediante el uso del parsing
- La tecnica es extraer toda una linea del archivo, convertirla en un flujo de datos, y obtener los datos relevantes.
 - La linea que se extrae mediante el getline se vuelve un flujo de datos, mediante istringstream, para luego poder subdividirla en los distintos datos extraibles.
 - Esto es mas conveniente cuando tenemos archivos tip CSV, en donde los datos estan separados por delimitadores

- Sin embargo, tambien se puede hacer la tecnica directa.

```
//=====LECTUA DE UN ARCHIVO DE TEXTO=====
#include <iostream>
#include <fstream>
#include <string>
#include <sstream>

int main(){
    std::ifstream file("registros.dat");

    if(!file.is_open()){
        std::cerr << "FILE NOT OPENED";
        return -1;
    }

    //Lectura de primera linea de encabezado -METODO DIRECTO
    std::string header;

    if(std::getline(file, header)){
        std::cout<<header<<std::endl;
    }

    //LECTURA DE CADA UNO DE LOS ARCHIVOS, TIPO CSV -METODO PARSING
    std::string line;

    while(std::getline(file, line)){
        std::istringstream str_line(line);
        std::string dat1;
        std::string dat2;

        std::getline(str_line, dat1, ',');
```

```
        std::getline(str_line, dat2, ',');

        std::cout<<"Data 1: "<<dat1<<"\n";
        std::cout<<"Data 2: "<<dat2<<"\n";
    }

    file.close();
    return 0;
}
```